

FARO — Seguridad, protocolos y manejo de datos

Libro 1 de 3 · Para la institución, el área de TI y el jurado

Versión: septiembre 2026 · Reto SHIFT 2026 · Gamificación

Para qué sirve este libro

Una universidad que conecta una herramienta a su Canvas está prestando algo valioso: el acceso a datos académicos de sus estudiantes. Este libro explica, sin adornos, qué hace FARO con ese acceso, qué protocolos usa, por qué esos protocolos son seguros y qué garantías se pueden **verificar** en lugar de creer.

Cada afirmación de este libro tiene una de estas tres marcas:

- **[código]** — está escrito en el código del repositorio y se cita el archivo.
- **[prueba]** — existe una prueba automática que lo verifica (`server/test/e2e.ts`).
- **[pendiente]** — es el diseño para producción y todavía no está implementado.

Si algo no tiene marca, es contexto.

Índice

1. Resumen ejecutivo
2. Arquitectura de confianza: dónde vive cada dato
3. Protocolos: por qué son seguros
4. El token de Canvas: ciclo de vida
5. Minimización y retención de datos
6. Modelo de amenazas y controles
7. Cumplimiento: LFPDPPP y políticas institucionales
8. Preguntas frecuentes para la universidad
9. Lista de verificación antes de un piloto

Las cinco garantías, en una página

Garantía	Cómo se cumple	Marca
FARO solo lee. No puede modificar calificaciones, entregas, fechas ni contenido.	Todas las llamadas son <code>GET</code> . El backend rechaza cualquier otro método antes de tocar la red. Los permisos LTI de escritura (<code>score</code> , <code>lineitem</code>) se rechazan explícitamente.	[código] [prueba]
La credencial nunca está en el navegador.	El token vive en <code>server/.env</code> y en la memoria del backend. La extensión no tiene ningún campo donde ponerlo y no tiene permiso de red hacia <code>instructure.com</code> .	[código] [prueba]
Solo seis rutas de Canvas, y nada más.	Una lista blanca en el backend (<code>server/src/allowlist.ts</code>) espejada en la extensión. Una ruta fuera de la lista se rechaza localmente, sin llamada a Canvas.	[código] [prueba]
El mentor de IA ve once campos y ningún dato personal.	El contexto se construye en un solo archivo (<code>src/lib/mentorContext.ts</code>) que no tiene acceso al nombre, correo, foto ni calificaciones.	[código]
Nunca se muestra un puntaje de riesgo al estudiante.	Los estados de fricción existen solo para decidir la intervención; la interfaz usa lenguaje de apoyo.	[código]

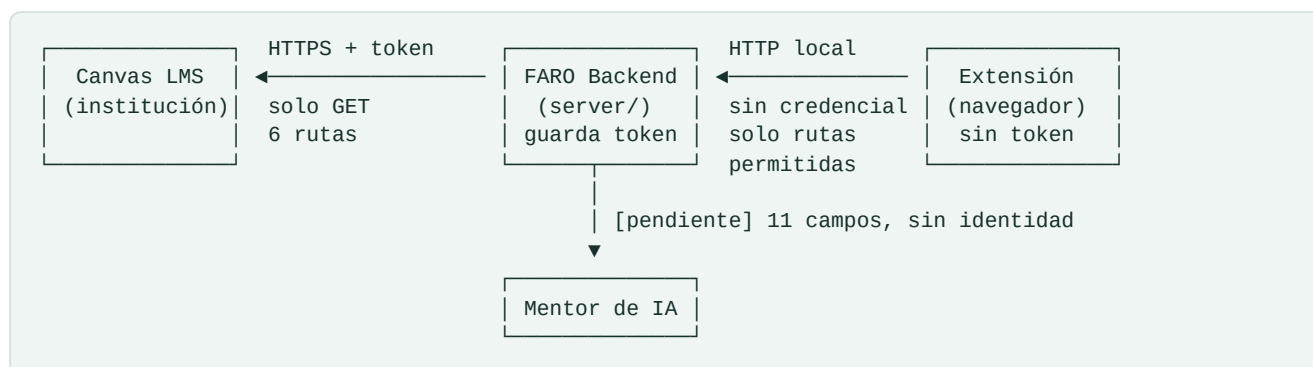
1. Resumen ejecutivo

Qué es FARO, en términos de datos

FARO es un acompañante de persistencia para estudiantes adultos que estudian en línea. Lee de Canvas la estructura del curso y el avance del estudiante, detecta cuándo alguien se está desconectando y le ofrece una ruta de regreso. No enseña el curso, no califica, no sustituye a Canvas.

Desde el punto de vista de la institución, FARO es **un lector de solo lectura** de una parte pequeña y bien definida de Canvas, con una capa de inteligencia artificial que recibe un contexto mínimo y anonimizado.

Las tres piezas y quién habla con quién



- **Canvas** sigue siendo la fuente de verdad académica. FARO no escribe ahí.
- **El backend** es el único proceso que conoce la credencial de Canvas. Es un servidor Node.js sin dependencias de terceros (`server/`), lo que reduce a cero el riesgo de cadena de suministro en el proceso que sostiene el token.
- **La extensión** es solo interfaz. No tiene token, no tiene permiso de red hacia Canvas, y solo puede pedir al backend rutas que el backend ya decidió permitir.
- **El mentor de IA** recibe un contexto construido a mano con once campos, ninguno de los cuales identifica a la persona.

Postura de seguridad en una frase

Reducir lo que FARO *puede* hacer hasta que coincida con lo que FARO *necesita* hacer, y hacer que esa reducción sea verificable por un administrador sin confiar en nosotros.

Estado actual, con honestidad

Componente	Estado
Backend con token, lista blanca, CORS restringido, límite de tasa, sin fuga de token en logs	Implementado y probado (20 verificaciones automáticas)
Extensión sin credenciales, modo <code>http</code> contra el backend	Implementado
Contrato de datos del mentor (11 campos)	Implementado
Pruebas contra un Canvas real (cuenta gratuita de Instructure)	Listo para ejecutar ; requiere un token del piloto
Lanzamiento LTI 1.3 firmado (OIDC + JWT)	Diseñado y documentado , rutas presentes que responden 501; requiere que la institución registre la herramienta
Token por estudiante vía OAuth2 (en vez de un token compartido)	Diseñado ; depende del registro LTI
Base de datos PostgreSQL con cifrado en reposo	Pendiente ; hoy el estado de FARO vive en el navegador del estudiante
Revisión de privacidad con la institución	Pendiente ; este libro es el insumo

2. Arquitectura de confianza: dónde vive cada dato

La pregunta que más importa en una revisión de seguridad no es "¿qué protocolo usan?" sino "¿dónde está cada dato y quién puede llegar a él?". Esta es la respuesta completa.

2.1 Zonas de confianza

FARO tiene cuatro zonas. Cada dato vive en exactamente una, y cruza una frontera solo por una razón documentada.

Zona	Qué contiene	Quién tiene acceso	Confianza
Z1 · Canvas	Todo el expediente académico	La institución	Máxima. FARO no la controla ni la altera.
Z2 · Backend FARO	El token de Canvas (en memoria), la configuración, un caché breve del lanzamiento	El operador del backend	Alta. Es el único proceso con credencial.
Z3 · Navegador del estudiante	Propósito, preferencias, sesiones registradas por FARO, perfil opcional, un instantáneo del curso mientras la pestaña está abierta	El estudiante	Media. Es su propio dispositivo.
Z4 · Servicio de IA	Once campos de contexto por conversación	El proveedor del modelo	Controlada por contrato de datos. Nunca recibe identidad.

2.2 Inventario de datos

Cada fila responde: qué es, de dónde sale, dónde se guarda, quién lo ve, cuánto dura.

Datos que vienen de Canvas (Z1 → Z2 → Z3)

Dato	Ruta de Canvas	Se guarda en	Retención
Nombre, código y fecha de cierre del curso	GET /api/v1/courses/:id	Memoria del navegador mientras FARO está abierto	Se descarta al cerrar

Dato	Ruta de Canvas	Se guarda en	Retención
Módulos, sus ítems y el estado de completitud del estudiante	GET /api/v1/courses/:id/modules?include[]=items	Igual	Igual
Actividades con la entrega del estudiante (fecha, estado, puntaje)	GET /api/v1/courses/:id/assignments?include[]=submission	Igual	Igual
Vistas de página por hora y participaciones	GET /api/v1/courses/:id/analytics/users/:uid/activity	Igual. Solo se conservan las marcas de tiempo; el conteo de vistas se descarta en el mapeo.	Igual
Cursos activos del estudiante	GET /api/v1/courses?enrollment_state=active	Backend, solo para elegir el curso cuando no está configurado	Un proceso
Id numérico del dueño del token	GET /api/v1/users/self	Backend, en caché. Nombre y avatar de esa respuesta se descartan.	Un proceso

[código] `src/data/canvas/endpoints.ts`, `src/data/canvas/client.ts` (función `mapActivity`), `server/src/routes/launch.ts`.

Datos que FARO crea (Z3)

Dato	Por qué existe	Se guarda en	Sale del navegador
Propósito (meta + frase de destino + minutos/día)	Personalizar la ruta y el mentor	<code>chrome.storage.local</code>	Meta, frase y minutos forman parte del contexto del mentor. Nada más.
Sesiones de estudio registradas por FARO	Impulso, logros, puntos	<code>chrome.storage.local</code>	No
Tarjetas de reconexión terminadas (fecha, ids de preguntas, aciertos a la primera)	Puntos y una oferta por día	<code>chrome.storage.local</code>	No. Las respuestas individuales no se guardan.
Canje de recompensa (nivel, fecha)	Un canje por semestre	<code>chrome.storage.local</code>	No en el prototipo (simulado). En producción, a la institución para entregar la recompensa.
Estado "pasó algo en mi vida"	Elegir la intervención	Memoria	No
Estilo del mentor, idioma, tema	Preferencias	<code>chrome.storage.local</code>	Estilo e idioma forman parte del contexto del mentor
Nombre, foto (reducida), bio	Perfil opcional en Comunidad	<code>chrome.storage.local</code>	No al mentor. En producción, a otros estudiantes del curso si el estudiante lo activa.
Conversación con el mentor	La sesión actual	Memoria	Hoy no: el mentor es local. En producción, cada mensaje irá al modelo con el contexto de once campos, a través del backend

[código] `src/state/storage.ts` , `src/state/store.tsx` , `src/types/index.ts` (comentarios de `Profile`).

Datos que nunca se recogen

Esta lista es tan importante como las anteriores. FARO **no** pide, lee ni almacena:

- Contraseñas de Canvas ni de la institución.
- Correo electrónico del estudiante.
- Lista de compañeros con nombre (no se llama al endpoint de inscripciones; **[prueba]** la ruta `/enrollments` se rechaza).
- Contenido de las entregas (textos, archivos, respuestas de exámenes).
- Calificaciones de otros estudiantes.
- Ubicación, dirección IP del estudiante (el backend registra solo la del cliente para el límite de tasa, en memoria, sin persistir).
- Historial de navegación fuera de las rutas de Canvas listadas.

2.3 Las fronteras y qué las cruza

Z1 Canvas → Z2 Backend	token del backend; respuesta completa de las 6 rutas
Z2 Backend → Z3 Navegador	JSON de Canvas ya paginado; NUNCA el token; NUNCA cabeceras de Canvas
Z3 Navegador → Z2 Backend	la ruta pedida (sin credencial); el backend la valida contra la lista
Z3 Navegador → Z2 → Z4 IA	[pendiente] 11 campos + el mensaje; NUNCA nombre, correo, foto, ids,

Lo que no aparece en el diagrama tampoco ocurre: la extensión no habla con Canvas (no tiene `host_permissions` para `instructure.com` [código] `public/manifest.json`), y hoy **ningún dato sale hacia un servicio de IA**: el mentor del prototipo es local y determinista (`LocalMentorService`), sin clave de API. La ruta `HttpMentorService` → `POST /api/mentor` ya existe en la extensión para cuando el backend incorpore la llamada al modelo (capítulo 5).

2.4 Por qué el backend no tiene dependencias

El proceso que sostiene la credencial institucional usa únicamente módulos nativos de Node.js (`node:http` , `node:crypto` , `node:fs`). No hay `node_modules` en producción. Esto no es minimalismo estético: cada paquete de terceros que corre dentro del proceso del token es código que la institución no escribió y que podría, en una actualización comprometida, leer el token de la memoria. Cero paquetes es cero superficie de ese tipo.

[código] `server/package.json` (`"dependencies": {}`).

3. Protocolos: por qué son seguros

FARO no inventa ningún mecanismo de seguridad. Usa los mismos estándares que Canvas exige a cualquier integración y los aplica de la forma más restrictiva que el producto permite. Este capítulo explica cada uno: qué problema resuelve, cómo lo usa FARO y qué garantiza.

3.1 TLS (HTTPS) — el canal

Problema que resuelve: que nadie en la red (Wi-Fi de una cafetería, un proveedor de internet, un proxy institucional mal configurado) pueda leer o alterar lo que viaja entre FARO y Canvas.

Cómo lo usa FARO: el backend se niega a arrancar si `CANVAS_API_URL` no empieza con `https://`. La única excepción es `localhost`, y existe solo para las pruebas automáticas contra el Canvas simulado.

[código] `server/src/env.ts`, función `loadEnv`.

Garantía: el token de Canvas solo viaja cifrado. Canvas, por su parte, solo acepta la API por HTTPS.

3.2 Token portador (Bearer) — la credencial de la API

Problema que resuelve: demostrar a Canvas quién hace la petición sin enviar una contraseña.

Cómo lo usa FARO: el backend añade `Authorization: Bearer <token>` a cada petición hacia Canvas, que es el método que Instructure recomienda ("If possible, using the HTTP Authorization header is recommended"). El token nunca va en la URL, nunca en el cuerpo, nunca en un log.

[código] `server/src/canvas.ts`, función `canvasGet`. **[prueba]** "the token was sent to Canvas on every call, and only there" y "log lines carry no query strings".

Garantía: Instructure describe los tokens como "password equivalent". FARO los trata exactamente así: viven en un solo proceso, en memoria, y se leen de un archivo que git ignora. El capítulo 4 detalla su ciclo de vida.

Limitación del piloto, declarada: un token personal generado a mano es lo que Instructure indica para pruebas ("For testing your application before you've implemented OAuth, the simplest option is to generate an access token on your user's profile page"). Para múltiples usuarios, la propia documentación es tajante: "Applications in use by multiple users MUST use OAuth to obtain tokens". FARO cumple eso con el camino LTI + OAuth2 de la sección 3.4. **[pendiente]**

3.3 Lista blanca de rutas — el alcance

Problema que resuelve: que un token con más permisos de los necesarios (un token personal de estudiante puede leer todo lo que el estudiante ve en Canvas) se use solo para lo que FARO

necesita.

Cómo lo usa FARO: el backend compara cada ruta pedida contra seis expresiones regulares. Solo `GET`. Solo los parámetros `include[]`, `per_page`, `enrollment_state` y `page`, con valores validados. Cualquier otra cosa se rechaza con `403` **antes de tocar la red**. La ruta se decodifica antes de compararla, así que `%2e%2e` no puede escapar del patrón.

[código] `server/src/allowlist.ts`. **[prueba]** "a path outside the allow list is refused locally (no Canvas call)", "an unknown query parameter is refused", "path traversal is refused", "a write is refused even on an allowed path".

Garantía: aunque la extensión fuera modificada por alguien, no puede convertir el backend en un proxy general hacia Canvas. Y aunque el token del piloto pudiera leer la lista de compañeros, FARO **no puede pedirla**.

3.4 LTI 1.3 + OpenID Connect + JWT — el lanzamiento institucional

Este es el camino de producción. Está diseñado y documentado en el código; las rutas existen y responden `501` hasta que la institución registre la herramienta. **[pendiente]**

Problema que resuelve: saber con certeza *qué curso* y *qué persona* están abriendo FARO, sin pedirle a nadie que se identifique y sin que FARO tenga que confiar en lo que la extensión dice.

Cómo funciona (IMS Security Framework 1.0):

1. Canvas inicia un *login* OpenID Connect hacia `GET /lti/login`. FARO guarda un `state` y un `nonce` de vida corta y redirige al endpoint de autorización de Canvas.
2. Canvas responde con `POST /lti/launch` y un `id_token`: un JWT **firmado por Canvas** con RS256.
3. FARO descarga las claves públicas de Canvas (JWKS), verifica la firma, el emisor (`iss`), la audiencia (`aud` = el `client_id` de FARO), la expiración y que el `nonce` sea el emitido y no se haya usado.
4. De las *claims* verificadas, FARO lee exactamente dos cosas: el curso (`https://purl.imsglobal.org/spec/lti/claim/context`) y la persona (`sub`).
5. FARO emite su propia sesión (cookie `HttpOnly`, `Secure`, `SameSite=None` porque corre dentro de un `iframe` de Canvas) ligada a ese curso y esa persona.

[código] `server/src/routes/lti.ts` (especificación completa en los comentarios).

Garantía: la identidad del estudiante la afirma Canvas criptográficamente, no la extensión. FARO no puede ser engañado para mostrar el curso de otra persona porque no acepta ninguna afirmación de identidad que no venga firmada por la institución.

Permisos LTI: FARO solicita `contextmembership.readonly` (NRPS, para conteos de presencia agregada) y `result.readonly` (AGS, resultados de solo lectura). **Rechaza** `score` y `lineitem`, los dos permisos que permitirían escribir en el libro de calificaciones. Un administrador puede verificar esto en la pantalla de la clave de desarrollador sin leer una línea de código.

[código] `src/data/canvas/endpoints.ts`, constante `ltiScopes`.

3.5 OAuth2 con clave de desarrollador — el token por estudiante

[pendiente] Una vez registrada la herramienta, cada estudiante obtiene su propio token de acceso mediante el flujo OAuth2 de Canvas, con los *scopes* limitados a las seis rutas (en el formato `url:GET|/api/v1/...` que Canvas usa en la clave de desarrollador). Los tokens de este flujo expiran en una hora y se renuevan con un *refresh token*, ambos guardados solo en el backend. Se revocan con `DELETE /login/oauth2/token`.

Garantía: desaparece el token compartido del piloto. Cada estudiante ve exactamente lo que Canvas le deja ver a él, y revocar el acceso de FARO a una persona es una operación de un solo token.

[código] los *scopes* ya están escritos en `src/data/canvas/endpoints.ts`, constante `requiredScopes`.

3.6 CORS — quién puede llamar al backend desde un navegador

Problema que resuelve: que una página web cualquiera, abierta en el mismo navegador, use el backend de FARO como si fuera la extensión.

Cómo lo usa FARO: el backend mantiene una lista de orígenes permitidos (`FARO_ALLOWED_ORIGINS`). Un origen fuera de la lista recibe `403` sin datos ni cabeceras CORS. Los métodos anunciados son solo `GET`, `OPTIONS`.

[código] `server/src/http.ts`, método `corsHeaders`. **[prueba]** "a browser origin outside the list gets 403 and no data", "CORS preflight succeeds for an allowed origin".

3.7 Límite de tasa — protección contra bucles

Problema que resuelve: una extensión con un error que reintenta en bucle podría enviar miles de peticiones a Canvas con el token institucional, y Canvas podría suspender ese token.

Cómo lo usa FARO: contador por cliente en ventana de un minuto (`FARO_RATE_LIMIT_PER_MINUTE`, 120 por defecto). Al superarlo, `429` con `Retry-After`.

[código] `server/src/http.ts`, clase `RateLimiter`. **[prueba]** "rate limit trips after the configured burst".

3.8 Cabeceras de respuesta

Toda respuesta del backend lleva `Cache-Control: no-store` (ningún dato académico queda en cachés intermedios), `X-Content-Type-Options: nosniff` y `Referrer-Policy: no-referrer`.

[código] `server/src/http.ts`, función `send`. **[prueba]** "responses carry no-store and nosniff".

3.9 Resumen: qué protocolo protege qué

Amenaza	Protocolo / control	Estado
Escucha en la red	TLS obligatorio	[código]
Robo de credencial desde el navegador	El navegador no tiene credencial	[código] [prueba]
Uso del token más allá de lo necesario	Lista blanca de 6 rutas, solo GET	[código] [prueba]
Suplantación de estudiante	LTI 1.3 con JWT firmado por Canvas	[pendiente]
Token compartido entre personas	OAuth2 por estudiante	[pendiente]
Página maliciosa usando el backend	CORS con lista de orígenes	[código] [prueba]
Bucle de peticiones	Límite de tasa	[código] [prueba]
Escritura en el expediente	Sin permisos de escritura; scopes <code>score / lineitem</code> rechazados	[código]

4. El token de Canvas: ciclo de vida

El token es el dato más sensible que FARO maneja. Este capítulo sigue su vida completa: dónde nace, dónde vive, por dónde viaja, dónde **no** está nunca, y cómo muere.

4.1 Nacimiento

Piloto (hoy): lo genera la persona dueña de la cuenta de Canvas, en *Cuenta* → *Configuración* → *Integraciones aprobadas* → *Nuevo token de acceso*. Canvas lo muestra una sola vez. Dos detalles de Canvas que juegan a favor:

- Las instituciones pueden **exigir fecha de expiración** para tokens de estudiantes (Instructure documenta un valor por defecto de 30 días para estudiantes y 90 para docentes), y pueden desactivar por completo la generación manual de tokens.
- El token hereda exactamente los permisos del usuario que lo creó: un token de estudiante no puede ver más que ese estudiante.

Producción: lo emite Canvas mediante OAuth2 a partir de una clave de desarrollador registrada por la institución, con *scopes* limitados y expiración de una hora. **[pendiente]**

4.2 Residencia

El token vive en **dos lugares y solo dos**:

1. `server/.env`, un archivo en la máquina que corre el backend. Está en `.gitignore` **[código]** y el repositorio incluye `server/.env.example` con el campo vacío para que nadie tenga que adivinar el formato.
2. La memoria del proceso del backend, en el objeto `Env`, desde el arranque hasta el apagado.

4.3 Viaje

Sale del backend en una única dirección: hacia Canvas, en la cabecera `Authorization`, por HTTPS, y solo para rutas de la lista blanca. Antes de seguir un enlace de paginación (`Link: rel="next"`), el backend verifica que la URL siga apuntando al host configurado de Canvas; un enlace que apuntara a otro dominio se rechaza, porque seguirlo enviaría el token fuera.

[código] `server/src/canvas.ts`, comprobación `url.startsWith(env.canvasApiUrl + '/')`.

4.4 Dónde no está nunca

Lugar	Por qué no	Verificación
La extensión	<code>CanvasConfig</code> no tiene campo para un token. No existe la variable.	<code>src/data/canvas/config.ts</code>
El build de Vite	Vite solo expone variables <code>VITE_*</code> ; el token no lleva ese prefijo y no está en <code>.env</code> de la raíz.	<code>.env.example</code> de la raíz lo dice explícitamente
Los logs del backend	El logger recibe método, ruta sin query, estado, duración e id. Nunca cabeceras.	[prueba] "the token was sent to Canvas on every call, and only there"
Las respuestas del backend	<code>describe(env)</code> es la única representación de la configuración que sale, y contiene <code>tokenPresent: true/false</code> , no el token.	[prueba] "health names the Canvas host and never the token"
Los mensajes de error	<code>CanvasError</code> guarda estado y ruta, nunca el cuerpo de la respuesta de Canvas.	<code>server/src/canvas.ts</code>
El repositorio git	<code>.gitignore</code> excluye <code>.env</code> en cualquier carpeta.	<code>.gitignore</code>
El servicio de IA	El contexto del mentor no tiene ningún campo de credencial.	<code>src/lib/mentorContext.ts</code>

4.5 Rotación

Cambiar el token es editar `server/.env` y reiniciar el backend. No hay caché del token en ningún otro lugar que sobreviva al reinicio. Recomendación para el piloto: tokens con expiración de 30 días, rotados al vencer.

4.6 Revocación

- **Desde Canvas:** la persona dueña del token lo elimina en *Integraciones aprobadas*. Desde ese instante el backend recibe `401` y lo reporta como `canvas_token_rejected` sin intentar nada más. **[prueba]** "a wrong token is reported as rejected, not as a generic failure".
- **Desde el backend:** borrar el valor en `.env` y reiniciar. El backend se niega a arrancar sin token, lo que hace imposible un estado "arrancó sin credencial y nadie se dio cuenta".
- **Producción (OAuth2):** `DELETE /login/oauth2/token` revoca el token del estudiante en Canvas. **[pendiente]**

4.7 Si el token se filtrara

Escenario: alguien obtiene acceso a `server/.env`.

1. **Alcance del daño:** lectura de lo que ese usuario de Canvas puede leer. No escritura: el token de un estudiante no puede alterar su expediente, y el de un docente tampoco a través de FARO, aunque sí directamente contra Canvas. Por eso el piloto usa **tokens de cuentas de estudiante**, nunca de docente ni de administrador.
2. **Contención:** revocar el token en Canvas (un clic). Generar uno nuevo. Reiniciar el backend.
3. **Detección:** Canvas registra el uso de cada token; un patrón de uso desde una IP desconocida es visible para la institución.
4. **Prevención estructural:** el camino LTI + OAuth2 elimina el archivo `.env` con token de larga vida y lo sustituye por tokens de una hora emitidos por estudiante.

5. Minimización y retención de datos

El principio rector de FARO en materia de datos es simple: **no recoger un dato porque es técnicamente posible, sino porque una función concreta lo necesita**. Este capítulo muestra cómo se aplica en cada capa, y qué pasa con los datos con el tiempo.

5.1 Minimización en la capa de Canvas

Canvas devuelve mucho más de lo que FARO usa. La capa de integración descarta lo que no necesita **en el momento de leerlo**, no después.

Lo que Canvas envía	Lo que FARO conserva	Lo que se descarta
Objeto de curso (decenas de campos)	id, nombre, código, fechas de inicio y fin	Todo lo demás: configuración, permisos, imagen, etc.
Módulos con ítems	id, nombre, posición, estado, conteo de ítems, <code>content_id</code> de cada ítem y si está completado	Títulos de páginas, URLs externas, archivos
Actividades con entrega	id, nombre, fecha límite, puntos posibles, y de la entrega: fecha, estado, puntaje	Descripción de la actividad, rúbrica, adjuntos, comentarios del docente
Actividad del estudiante	Solo marcas de tiempo. El número de vistas por hora se elimina.	Conteos de vistas, URLs visitadas
<code>users/self</code>	El id numérico	Nombre, avatar, correo, zona horaria

[código] `src/data/canvas/raw.ts` declara solo los campos leídos; `src/data/canvas/client.ts` hace el mapeo; `server/src/routes/launch.ts` conserva únicamente `self.data.id`.

Por qué importa el caso de la actividad: "cuántas páginas vio a las 3 de la tarde" es un dato de vigilancia. "Cuándo fue la última vez que entró" es el único dato que el motor de fricción necesita. FARO conserva el segundo y destruye el primero.

5.2 Minimización en el mentor de IA

El contexto que recibe el modelo se construye en una sola función y contiene exactamente estos once campos:

```
{
  "course": "Gestión de Proyectos",
  "progress": 44,
  "next_activity": "Caso de estudio: alcance",
  "estimated_time": 25,
  "student_goal": "career_growth",
  "destination": "Liderar mis propios proyectos",
  "available_time": 20,
  "momentum": 62,
```

```
"friction_state": "FRICTION",
"style": "encouraging",
"language": "es"
}
```

No hay nombre, correo, id de Canvas, calificaciones, historial de otras conversaciones ni clasificación de riesgo de abandono. El estado de fricción es una de cinco etiquetas de intervención (`FLOWING` , `FRICTION` , `POSSIBLE_OVERWHELM` , `DISCONNECTION` , `RECOVERY`), no un puntaje.

La única información de origen humano es la **frase de destino**, que el estudiante escribe libremente al iniciar. Se le muestra el ejemplo *"Liderar mis propios proyectos en vez de solo ejecutarlos"* para orientar hacia una meta, no hacia datos personales. En producción, esta frase debe pasar por una advertencia explícita: *"no incluyas datos que te identifiquen"*. **[pendiente]**

[código] `src/lib/mentorContext.ts` , `src/types/index.ts` (interfaz `MentorContext`).

5.3 Minimización en Comunidad

Comunidad es la parte de FARO con más tentación de recoger datos sociales, y por eso tiene las reglas más estrictas, escritas en los tipos:

- **No existe campo de seguidores ni de ranking.** No es que estén ocultos: no están en el modelo de datos, así que ninguna función puede calcularlos.
- **La presencia es agregada.** "23 estudiantes estudiando ahora" es un conteo. "Andrea lleva 37 minutos" nunca aparece, y el tipo `CoursePresence` solo admite números.
- **Los compañeros no tienen foto.** Solo iniciales y un color decorativo. Únicamente el propio estudiante puede añadir una imagen a su avatar, reducida en su navegador antes de guardarse.
- **Las reacciones se muestran por separado**, nunca sumadas en un marcador.

[código] `src/types/index.ts` , comentarios de `CommunityAuthor` y `CoursePresence` .

5.4 Retención

Dato	Dónde	Cuánto dura	Cómo se borra
Instantáneo del curso (de Canvas)	Memoria del navegador	Mientras FARO está abierto	Al cerrar la pestaña o el panel
Registro de peticiones a Canvas	Memoria del navegador, 30 entradas	Mientras FARO está abierto	Automático; nunca se persiste ni se envía
Propósito, preferencias, sesiones, perfil	<code>chrome.storage.local</code>	Hasta que el estudiante lo borre	Botón <i>Restablecer</i> en Progreso, o desinstalar la extensión
Token de Canvas	<code>server/.env</code> + memoria del backend	Hasta rotación o revocación	Editar <code>.env</code> y reiniciar; revocar en Canvas

Dato	Dónde	Cuánto dura	Cómo se borra
Caché del lanzamiento (id de curso, id de usuario)	Memoria del backend	Hasta reiniciar el proceso	Reiniciar
Logs del backend	Salida estándar	Los define el operador	Los define el operador; no contienen datos personales ni query strings

Producción [pendiente]: cuando exista PostgreSQL, se propone:

- Sesiones de estudio, propósito y preferencias: mientras la inscripción esté activa, más 90 días para permitir "regresar" a un curso pausado; después, borrado automático.
- Conversaciones con el mentor: no se almacenan más allá de la sesión, salvo consentimiento explícito para mejorar el producto, en cuyo caso se anonimizan.
- Eventos de fricción e intervenciones: agregados para métricas del piloto, con el id del estudiante sustituido por un id pseudónimo que solo la institución puede resolver.

5.5 Lo que el estudiante puede ver y controlar

- **Perfil** → **Canvas** muestra en vivo cada llamada que FARO hace, con su ruta, estado y latencia. La transparencia no es un documento: es una pantalla.
- **Perfil** → **Canvas** muestra los permisos LTI que FARO rechaza.
- **FARO Mentor** muestra, al pie, la lista exacta de lo que el mentor sabe y lo que no.
- **Progreso** → **Restablecer** borra todo el estado local de FARO.
- Nada del perfil es obligatorio. FARO funciona completo sin nombre ni foto.

6. Modelo de amenazas y controles

Un modelo de amenazas responde tres preguntas: qué queremos proteger, de quién, y qué hemos hecho al respecto. Esta tabla es la respuesta, amenaza por amenaza. La columna *Prueba* nombra la verificación automática en `server/test/e2e.ts` cuando existe.

6.1 Activos

1. **El token de Canvas** — da acceso de lectura al expediente del dueño.
2. **Los datos académicos del estudiante** — avance, entregas, fechas.
3. **La identidad del estudiante** — quién es la persona detrás de una sesión de FARO.
4. **La integridad del expediente** — que nada de lo que FARO hace pueda alterar Canvas.
5. **La confianza del estudiante** — que FARO no se convierta en vigilancia.

6.2 Amenazas y controles

#	Amenaza	Activo	Control	Estado	Prueba
T1	Un atacante extrae la credencial del código de la extensión (las extensiones son código público)	Token	No hay credencial en la extensión. <code>CanvasConfig</code> no tiene campo para ella.	[código]	—
T2	Una extensión modificada usa el backend para leer cualquier ruta de Canvas	Datos	Lista blanca de 6 rutas en el backend; <code>403</code> local sin llamada a Canvas	[código]	"a path outside the allow list is refused locally"
T3	Una extensión modificada intenta escribir en Canvas a través del backend	Integridad	Solo <code>GET</code> ; <code>405</code> para otros métodos; sin scopes de escritura	[código]	"a write is refused even on an allowed path"
T4	Manipulación de parámetros para ampliar la respuesta de Canvas (<code>all_dates</code> , <code>include[]=user</code>)	Datos	Parámetros y valores validados; el resto se rechaza	[código]	"an unknown query parameter is refused"
T5	<i>Path traversal</i> codificado (<code>..%2F</code>) para salir del patrón	Datos	Decodificación antes de comparar; <code>..</code> y <code>//</code> rechazados	[código]	"path traversal is refused"

#	Amenaza	Activo	Control	Estado	Prueba
T6	Una página web maliciosa en el mismo navegador llama al backend	Datos	CORS con lista de orígenes; origen desconocido → 403 sin datos	[código]	"a browser origin outside the list gets 403"
T7	Bucle de peticiones agota la cuota de Canvas y expone el token a suspensión	Token	Límite de tasa por cliente, 429	[código]	"rate limit trips after the configured burst"
T8	El token aparece en logs, errores o respuestas	Token	Logger sin cabeceras ni query; errores sin cuerpo de Canvas; describe() sin token	[código]	"health names the Canvas host and never the token", "log lines carry no query strings"
T9	Un Link de paginación manipulado redirige el token a otro dominio	Token	El backend verifica que cada página siga en el host de Canvas configurado	[código]	—
T10	Escucha de red entre backend y Canvas	Token, datos	HTTPS obligatorio; el backend no arranca con http:// fuera de localhost	[código]	—
T11	Suplantación: alguien afirma ser otro estudiante	Identidad	Piloto: el backend deduce la identidad del token, no de la extensión. Producción: JWT firmado por Canvas (LTI 1.3)	[código] / [pendiente]	"launch resolves the token owner"
T12	Cachés intermedios conservan datos académicos	Datos	Cache-Control: no-store en toda respuesta	[código]	"responses carry no-store and nosniff"
T13	Un paquete de terceros comprometido lee el token de la memoria del backend	Token	Cero dependencias en el backend	[código]	—
T14	El servicio de IA recibe datos personales	Identidad, confianza	Contexto de 11 campos construido en una sola función; sin nombre, correo, ids, calificaciones	[código]	—
T15	FARO se convierte en herramienta de vigilancia (quién estudia cuánto)	Confianza	Presencia agregada en el modelo de datos; sin ranking ni seguidores; conteos de vistas descartados	[código]	—

#	Amenaza	Activo	Control	Estado	Prueba
T16	Se expone al estudiante una clasificación de riesgo de abandono	Confianza	Los estados de fricción son internos; la interfaz usa lenguaje de apoyo; no existe puntaje de riesgo	[código]	—
T17	Un token de docente o administrador en el piloto amplía el alcance de una fuga	Token, datos	Política: el piloto usa únicamente tokens de cuentas de estudiante	Política	—
T18	El backend arranca sin credencial y sirve datos vacíos como si fueran reales	Confianza	Se niega a arrancar sin <code>CANVAS_ACCESS_TOKEN</code> ; con token inválido responde <code>canvas_token_rejected</code>	[código]	"a wrong token is reported as rejected"
T19	Una cuenta de estudiante no puede leer analíticas y FARO inventa actividad	Confianza	<code>401 / 403</code> se pasa al cliente; FARO usa fechas de entregas y lo dice en pantalla	[código]	"Canvas 401 on analytics is passed through"
T20	Robo del archivo <code>server/.env</code>	Token	Alcance limitado a lectura de un estudiante; revocación de un clic en Canvas; el camino OAuth2 elimina el archivo	Política / [pendiente]	—

6.3 Lo que este modelo no cubre todavía

- **Autenticación de la extensión ante el backend en producción.** Hoy el backend confía en el origen (CORS) y en escuchar solo en `127.0.0.1` . En un despliegue institucional, la sesión emitida tras el lanzamiento LTI es lo que autoriza cada llamada. **[pendiente]**
- **Cifrado en reposo** de la futura base de datos. **[pendiente]**
- **Auditoría de accesos** por parte de la institución: quién consultó qué y cuándo. El backend registra ruta y estado por petición; falta la persistencia y el id pseudónimo. **[pendiente]**
- **Revisión de seguridad externa.** Este documento es autoevaluación; un piloto institucional debe incluir una revisión por el área de seguridad de la universidad.

7. Cumplimiento: LFPDPPP y políticas institucionales

Este capítulo es orientación técnica para preparar la revisión legal de la institución. No sustituye la opinión del área jurídica ni del responsable de protección de datos de la universidad.

7.1 El marco aplicable en México

La **Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP)** fue publicada en el Diario Oficial de la Federación el 20 de marzo de 2025 y entró en vigor el 21 de marzo de 2025. Sustituye a la ley de 2010. La autoridad reguladora para el sector privado ya no es el INAI, disuelto, sino la **Secretaría Anticorrupción y Buen Gobierno**.

Tres puntos de la nueva ley que tocan directamente a FARO:

- 1. **Definición de dato personal.** Es toda información de una persona identificada o *identificable*, "cuando su identidad pueda determinarse directa o indirectamente a través de cualquier información". Un id numérico de Canvas es, por tanto, dato personal aunque no lleve nombre: es indirectamente identificable por la institución.
- 2. **Aviso de privacidad.** Debe identificar datos sensibles y distinguir las finalidades que requieren consentimiento expreso. Cuando los datos se recaban por medios electrónicos, debe ofrecerse un aviso simplificado que remita al completo.
- 3. **Consentimiento.** Puede ser expreso o tácito; es tácito cuando, puesto a disposición el aviso de privacidad, el titular no manifiesta lo contrario. Para datos sensibles sigue exigiéndose consentimiento expreso.

Nota sobre el sujeto obligado. Una universidad privada como Tecmilenio es un particular y le aplica la LFPDPPP. Una universidad pública se rige por la Ley General de Protección de Datos Personales en Posesión de Sujetos Obligados. Ambas comparten los principios; el análisis de este capítulo es válido para las dos, pero el instrumento y la autoridad cambian. **La institución debe confirmar cuál le aplica.**

7.2 Los principios, uno por uno

Principio	Qué exige	Cómo lo cumple FARO	Marca
Licitud	Tratar datos conforme a la ley y sin engaño	FARO solo lee lo que el estudiante ya puede ver de sí mismo en Canvas; el registro de peticiones es visible en pantalla	[código]
Consentimiento	Contar con la voluntad del titular	Piloto: consentimiento expreso e informado de cada participante (plantilla en 7.4). Producción: aviso de privacidad integrado en el lanzamiento LTI	Política / [pendiente]

Principio	Qué exige	Cómo lo cumple FARO	Marca
Información	Aviso de privacidad accesible	Pantalla <i>Perfil</i> → <i>Canvas</i> y pie del Mentor muestran qué se consulta y qué se envía; aviso formal por redactar con la institución	[código] / [pendiente]
Calidad	Datos exactos y actualizados	Canvas es la fuente de verdad; FARO no copia el expediente, lo lee en cada apertura	[código]
Finalidad	Usar los datos solo para lo informado	Cada ruta de Canvas tiene un propósito escrito junto a ella (<code>server/src/allowlist.ts</code> , campo <code>purpose</code>); no hay uso secundario	[código]
Lealtad	No obtener datos por medios engañosos	No hay rastreo fuera de las 6 rutas; no hay <i>dark patterns</i> para obtener el perfil (todo es opcional)	[código]
Proporcionalidad	Solo los datos necesarios	Capítulo 5: conteos de vistas descartados, nombre y avatar de <code>users/self</code> descartados, mentor con 11 campos	[código]
Responsabilidad	Velar por el cumplimiento y poder demostrarlo	Este libro; las 20 pruebas automáticas; la lista de verificación del capítulo 9	[prueba]

7.3 Datos sensibles

FARO **no trata datos sensibles** según la definición de la ley (origen étnico, salud, creencias, opiniones políticas, preferencia sexual, datos genéticos). Dos matices que la institución debe considerar:

- El estado *"pasó algo en mi vida"* (menos tiempo, abrumado, necesito una pausa) es autodeclarado, se usa solo para elegir una intervención, **no se persiste ni sale del navegador**. No es un dato de salud, pero el aviso de privacidad debe mencionarlo para no dejar dudas.
- La **frase de destino** es texto libre. Si un estudiante escribiera ahí un dato sensible, FARO lo guardaría localmente y lo incluiría en el contexto del mentor. Mitigación: advertencia en pantalla [pendiente] y política de no almacenar conversaciones.

7.4 Consentimiento para el piloto

Para un piloto con cinco a treinta estudiantes, se recomienda consentimiento expreso por escrito. Elementos mínimos que debe contener:

1. Quién es el responsable (la institución) y quién el encargado (el equipo de FARO).
2. Qué datos se leen de Canvas (las seis rutas, en lenguaje llano).
3. Qué datos crea FARO (propósito, sesiones, preferencias, perfil opcional).
4. Que FARO no modifica nada en Canvas.
5. Que el mentor de IA, cuando se active, recibirá un contexto sin nombre ni correo.

6. Cuánto tiempo se conservan los datos y cómo pedir su borrado (*Progreso* → *Restablecer*, y revocación del token).
7. Que la participación es voluntaria y retirarse no afecta la calificación.
8. Cómo ejercer derechos ARCO ante la institución.

7.5 Derechos ARCO

Derecho	Cómo se ejerce en FARO hoy	En producción
Acceso	<i>Perfil</i> muestra todo lo que FARO guarda del estudiante; <i>Perfil</i> → <i>Canvas</i> muestra todo lo que lee	Exportación desde el backend
Rectificación	Los datos académicos se corrigen en Canvas (fuente de verdad); los de FARO, en Perfil	Igual
Cancelación	<i>Progreso</i> → <i>Restablecer</i> borra el estado local; desinstalar la extensión lo elimina todo	Borrado en base de datos por solicitud
Oposición	No instalar la extensión; retirar el consentimiento del piloto	Desactivar la herramienta LTI para ese estudiante

7.6 Transferencias

- **A Canvas (Instructure):** FARO no transfiere nada a Canvas; solo lee. La relación de la institución con Instructure ya está cubierta por su contrato existente.
- **Al proveedor del modelo de IA [pendiente]:** será una transferencia de los 11 campos del contexto más el mensaje del estudiante. Debe constar en el aviso de privacidad, y el contrato con el proveedor debe prohibir el uso de esos datos para entrenamiento. Hasta entonces, el mentor es local y no hay transferencia.
- **Entre dispositivos del estudiante [pendiente]:** cuando exista el backend con base de datos, el estado de FARO se sincronizará a través de él. Es tratamiento por el encargado, no transferencia a tercero.

7.7 Políticas institucionales de Canvas

Independientemente de la ley, la institución controla en Canvas:

- Si los estudiantes pueden generar tokens de acceso y con qué expiración.
- Qué *scopes* concede a la clave de desarrollador de FARO (recomendación: exactamente los de `requiredScopes`).
- Si la herramienta LTI se instala a nivel de cuenta, subcuenta o curso.
- Si los estudiantes pueden ver sus propias analíticas (afecta si FARO usa el endpoint de actividad o el respaldo por entregas).

Ninguna de estas decisiones requiere cambios en FARO; el backend respeta lo que Canvas permita y reporta lo que no.

8. Preguntas frecuentes para la universidad

Las preguntas que un área de TI, un responsable de datos o un directivo académico hacen antes de conectar una herramienta a Canvas, con respuestas cortas y la referencia para comprobarlas.

Acceso y alcance

¿FARO puede modificar calificaciones, entregas o fechas? No. Todas las llamadas a Canvas son de lectura (`GET`). El backend rechaza cualquier otro método antes de contactar a Canvas, y los permisos LTI que permitirían escribir en el libro de calificaciones (`score` , `lineitem`) se rechazan explícitamente. Se comprueba en `server/src/allowlist.ts` y en la pantalla de la clave de desarrollador de Canvas.

¿Qué ve exactamente FARO de Canvas? Seis rutas: el curso, sus módulos con ítems, las actividades con la entrega del propio estudiante, la actividad (marcas de tiempo) del propio estudiante, la lista de cursos activos del estudiante y su id numérico. Nada más. La lista está en un archivo de treinta líneas: `server/src/allowlist.ts` .

¿Ve FARO los datos de otros estudiantes? No. No llama al endpoint de inscripciones ni al de usuarios del curso. En producción, el conteo de "cuántos están estudiando" vendrá del servicio NRPS de LTI en modo agregado, sin nombres.

¿Puede FARO ver el contenido de una entrega o de un examen? No. Lee fecha, estado y puntaje de la entrega, no su contenido. Los campos que se leen están declarados en `src/data/canvas/raw.ts` ; lo que no está ahí, se ignora.

¿Un estudiante puede usar FARO para ver el curso de otro? No. En el piloto, el backend deduce la identidad del token, no de lo que la extensión afirma. En producción, la identidad la firma Canvas (LTI 1.3) y FARO no acepta otra.

Credenciales

¿Dónde está el token de Canvas? En `server/.env` , en la máquina que corre el backend, y en la memoria de ese proceso. En ningún otro lugar. La extensión no tiene ningún campo para un token y no tiene permiso de red hacia `instructure.com` .

¿El token está en el código fuente o en GitHub? No. `.env` está en `.gitignore` . El repositorio contiene `server/.env.example` con el campo vacío.

¿Qué pasa si el token se filtra? Se revoca en Canvas en un clic, se genera otro, se reinicia el backend. El alcance del daño es lectura de lo que ese estudiante ve de sí mismo. Por eso el piloto usa solo tokens de cuentas de estudiante, nunca de docente ni de administrador. Capítulo 4.

¿Cada estudiante necesita generar un token? En el piloto, sí, uno por participante (o un token de una cuenta de prueba). Es la práctica que Instructure indica para pruebas. Para un despliegue real,

FARO usa OAuth2 con clave de desarrollador y los tokens los emite Canvas automáticamente, con una hora de vida.

Inteligencia artificial

¿Qué recibe el modelo de IA? Once campos: nombre del curso, porcentaje de avance, siguiente actividad y su duración, meta declarada, frase de destino, minutos disponibles, impulso, estado de fricción, estilo e idioma. Está en `src/lib/mentorContext.ts`. No recibe nombre, correo, ids, calificaciones ni historial.

¿El modelo puede hacer la tarea del estudiante? No. Hay un filtro de peticiones del tipo "hazme el examen" que responde con una negativa y ofrece dividir la tarea, explicar el concepto o revisar el razonamiento. El filtro es igual en los cinco estilos de comunicación.

¿Hoy los datos van a un proveedor de IA? No. El mentor del prototipo es local y determinista. La llamada al modelo se activará desde el backend cuando exista contrato con el proveedor y aviso de privacidad actualizado.

¿Se usan los datos para entrenar modelos? No, y el contrato con el proveedor deberá prohibirlo explícitamente antes de activar el mentor remoto.

Vigilancia y bienestar

¿FARO calcula un "riesgo de abandono" del estudiante? Calcula estados de fricción internos (cinco etiquetas basadas en días sin actividad y trabajo pendiente) para decidir qué intervención ofrecer. **Nunca** muestra un puntaje de riesgo al estudiante y no lo comparte con nadie. La interfaz dice "tu ruta necesita atención", no "riesgo: 87%".

¿Puede un docente ver quién estudia cuánto? No existe esa pantalla, y el modelo de datos de Comunidad solo admite conteos agregados, no personas. Si la institución quisiera métricas del piloto, se propone entregarlas agregadas y con ids pseudónimos.

¿FARO envía notificaciones o recordatorios? No. No hay notificaciones push, correos ni recordatorios. FARO espera a que el estudiante abra el panel. El "puerto seguro" (pausa planeada) explícitamente no tiene cuenta regresiva.

¿Es FARO un videojuego? No. Usa mecánicas (ruta, puntos de control, logros) sin vidas, niveles ni tablas de posiciones. Los cinco logros y cinco reconocimientos exigen haber persistido; ninguno se gana haciendo clic.

Operación

¿Dónde corre el backend? En el piloto, en una máquina del equipo o de la institución, escuchando en `127.0.0.1`. En producción, en infraestructura de la institución o en un proveedor aprobado, detrás de TLS, con el lanzamiento LTI como autenticación.

¿Qué dependencias tiene? El backend, ninguna. Solo Node.js. La extensión, dos: `react` y `react-dom`.

¿Cómo sé que las garantías de este libro son ciertas? Ejecutando `npm test` dentro de `server/`: veinte verificaciones automáticas contra un Canvas simulado con paginación real. Y abriendo *Perfil* → *Canvas* en FARO: el registro de peticiones en vivo muestra cada llamada.

¿Qué falta para producción? Registro LTI 1.3 por la institución, OAuth2 por estudiante, base de datos con cifrado en reposo, mentor remoto con contrato de datos, aviso de privacidad formal y revisión de seguridad externa. Capítulo 9 tiene la lista completa.

¿FARO reemplaza a Canvas? No. Canvas sigue siendo la fuente de verdad académica. FARO lee de Canvas y añade una capa de acompañamiento. *No reemplazamos Canvas: lo hacemos adaptativo.*

9. Lista de verificación antes de un piloto

Marcar cada punto antes de conectar FARO a un Canvas con estudiantes reales. Los puntos con ⚙ los verifica el equipo técnico; los puntos con 🏛 los verifica la institución.

A. Antes de arrancar el backend

- ☐ ⚙ `server/.env` existe, **no** está en git (`git status` no lo lista) y `CANVAS_API_URL` empieza con `https://`.
- ☐ ⚙ El token pertenece a una **cuenta de estudiante** del piloto, no a un docente ni administrador.
- ☐ ⚙ El token tiene **fecha de expiración** (recomendado: 30 días).
- ☐ ⚙ `FARO_HOST=127.0.0.1` salvo que haya TLS por delante.
- ☐ ⚙ `FARO_ALLOWED_ORIGINS` contiene solo los orígenes reales (el id de la extensión cargada, el servidor de desarrollo si aplica).
- ☐ ⚙ `npm test` en `server/` pasa las 20 verificaciones.
- ☐ ⚙ `curl http://127.0.0.1:3000/health` responde con el host de Canvas y **no** contiene el token.

B. Antes de cargar la extensión

- ☐ ⚙ Se construyó con `VITE_CANVAS_MODE=http` y `VITE_FARO_API_URL` apuntando al backend.
- ☐ ⚙ `public/manifest.json` **no** contiene `instructure.com` en `host_permissions`.
- ☐ ⚙ En *Perfil* → *Canvas* la insignia dice *En vivo* y el registro muestra estado `200` en las cuatro rutas.
- ☐ ⚙ Si aparece la nota "tu cuenta no puede leer las analíticas", se decidió con la institución si se habilita el permiso o se acepta el respaldo por entregas.

C. Con la institución

- ☐ 🏛 Se confirmó qué ley aplica (LFPDPPP para institución privada; Ley General para pública).
- ☐ 🏛 Existe un aviso de privacidad (simplificado + completo) revisado por jurídico, que menciona: las seis rutas de Canvas, los datos que FARO crea, el estado "pasó algo en mi vida", el futuro mentor remoto y la retención.
- ☐ 🏛 Cada participante firmó el consentimiento del piloto (capítulo 7.4).
- ☐ 🏛 Se acordó cómo se ejercen los derechos ARCO durante el piloto y quién responde.
- ☐ 🏛 Se acordó qué métricas del piloto se entregan y en qué forma (agregadas, con ids pseudónimos).
- ☐ 🏛 El área de seguridad de la institución revisó este libro y el código de `server/`.

- ☐ ⚙ Se definió la fecha de fin del piloto y el procedimiento de revocación de tokens y borrado de datos al terminar.

D. Al terminar el piloto

- ☐ ⚙ Todos los tokens del piloto revocados en Canvas.
- ☐ ⚙ `server/.env` borrado de la máquina del backend.
- ☐ ⚙ Participantes informados de cómo borrar el estado local (*Progreso* → *Restablecer* o *desinstalar*).
- ☐ ⚙ Métricas entregadas; datos crudos destruidos o entregados a la institución según lo acordado.

E. Para pasar a producción (fuera del alcance del piloto)

- ☐ Registro de FARO como herramienta LTI 1.3 a nivel de cuenta en Canvas, con los scopes exactos de `requiredScopes` y sin `score` ni `lineitem`.
- ☐ Implementación de `/lti/login`, `/lti/launch` y `/.well-known/jwks.json` según la especificación en `server/src/routes/lti.ts`.
- ☐ OAuth2 por estudiante; eliminación del token compartido.
- ☐ PostgreSQL con cifrado en reposo; política de retención automática.
- ☐ Mentor remoto tras el backend, con contrato de datos que prohíba entrenamiento.
- ☐ Revisión de seguridad externa.